Reviewer Pack — Minimal Coherence Length of Braided Categories and SAT Claus…

Entry 499b999bfe8f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 11:22:41 UTC

Minimal Coherence Length of Braided Categories and SAT Clause Complexity

Entry ID: `499b999bfe8f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 11:22:41 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Braided Categories) **Field B** (complexity object): Boolean Satisfiability (Clause Complexity)

Statement:

> For every Boolean satisfiability instance φ with n clauses, the minimal coherence length c(φ) of its associated braided category is linearly correlated with its clause complexity κ(φ), such that c(φ) = Θ(κ(φ)).

Rationale (proposer's reasoning):

> Braided categories provide a categorical framework for studying non-commutative structures, and their coherence length measures the degree of non-commutativity. Since SAT clause complexity captures the difficulty of determining satisfiability, there may be an underlying connection between the two that reveals new insights into the computational nature of Boolean satisfiability problems.

Taxonomy category: `BraidedCategories` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b2c1784487286dd1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For each Boolean satisfiability instance φ, if the linear regression coefficient of the correlation between c(φ) and κ(φ) is greater than or equal to 0.8 and the mean absolute error is less than or equal to 3, then support for the conjecture is provided. Otherwise, if any seed produces a coefficient less than 0.8 or a mean absolute error greater than 3, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'braided categories' AND 'SAT clause complexity' - 'coherence length' INBRACKET braided categories AND 'Boolean satisfiability' - 'Category Theory' AND 'minimal coherence length' AND 'clause complexity'

Top relevant hits considered: - [http://arxiv.org/abs/1807.11061v1] Clause Vivification by Unit Propagation in CDCL SAT Solvers - [http://arxiv.org/abs/1805.01774v2] A Tangent Category Alternative to the Faà di Bruno Construction - [http://arxiv.org/abs/math/0601386v2] Thin fillers in the cubical nerves of omega-categories - [http://arxiv.org/abs/1808.09738v4] A characterisation of the category of compact Hausdorff spaces - [http://arxiv.org/abs/0801.3124v3] Coherence length of cosmic background radiation enlarges the attenuation length of the ultra-high energy proton - [http://arxiv.org/abs/2403.01651v3] Dagger n-categories - [http://arxiv.org/abs/2208.02745v3] A homotopy coherent nerve for (∞, n)-categories - [http://arxiv.org/abs/1005.0209v2] Approximations and adjoints in homotopy categories

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(n):
        clauses = []
        for _ in range(n):
            literals = [random.choice(['A', 'B', 'C']) + ('' if random.random() < 0.5 else "'") fo
            clause = f"({' & '.join(literals)})"
            clauses.append(clause)
        return " | ".join(clauses)

    def compute_clause_complexity(formula):
        return formula.count(" | ") + 1

    def construct_braided_category(formula):
        # Placeholder for the actual construction of the braided category
        # This is a dummy implementation that returns a constant value
```

```
        return 5.0

    def linear_regression(x, y):
        n = len(x)
        if n == 0:
            return 0, 0
        sum_x = sum(x)
        sum_y = sum(y)
        sum_xy = sum(xi * yi for xi, yi in zip(x, y))
        sum_xx = sum(xi ** 2 for xi in x)
        slope = (n * sum_xy - sum_x * sum_y) / (n * sum_xx - sum_x ** 2)
        intercept = (sum_y - slope * sum_x) / n
        return slope, intercept

    def mean_absolute_error(y_true, y_pred):
        return sum(abs(yt - yp) for yt, yp in zip(y_true, y_pred)) / len(y_true)

    instances_tested = 0
    c_values = []
    kappa_values = []

    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):
            formula = generate_formula(n)
            kappa = compute_clause_complexity(formula)
            c = construct_braided_category(formula)
            if c is None or kappa is None:
                continue
            instances_tested += 1
            c_values.append(c)
            kappa_values.append(kappa)

    if instances_tested == 0:
        return {
            "metric_name": "coherence_length",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    slope, _ = linear_regression(kappa_values, c_values)
    mae = mean_absolute_error(kappa_values, c_values)

    return {
        "metric_name": "coherence_length",
        "metric_value": slope,
        "instances_tested": instances_tested,
        "n_max": max(40, n),
        "conjecture_holds": slope >= 0.8 and mae <= 3,
        "counterexample": "" if slope >= 0.8 and mae <= 3 else f"slope={slope}, mae={mae}"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
```

```
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_slope = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_slope = math.sqrt(sum((r["metric_value"] - mean_slope) ** 2 for r in results if r["metric_
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_slope} std={std_slope} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"slope={result['counterexample']}\" first_failin
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
e_holds': False, 'counterexample': 'slope=0.0, mae=15.0'}
TRIAL: {'metric_name': 'coherence_length', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 40,
TRIAL: {'metric_name': 'coherence_length', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 40,
TRIAL: {'metric_name': 'coherence_length', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 40,
TRIAL: {'metric_name': 'coherence_length', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 40,
TRIAL: {'metric_name': 'coherence_length', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 40,
TRIAL: {'metric_name': 'coherence_length', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 40,
TRIAL: {'metric_name': 'coherence_length', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 40,
TRIAL: {'metric_name': 'coherence_length', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 40,
TRIAL: {'metric_name': 'coherence_length', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 40,
TRIAL: {'metric_name': 'coherence_length', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 40,
TRIAL: {'metric_name': 'coherence_length', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 40,
RESULT: FALSIFIED counterexample="slope=slope=0.0, mae=15.0" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The test code provides a placeholder for the construction of the braided category, which is not implemented correctly. The `construct_braided_category` function returns a constant value instead of constructing an actual braided category associated with the formula. This invalidates the empirical test as it does not measure the minimal coherence length c(φ) of the associated braided category.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that the conjecture does not hold for any of the tested instances, as the linear regression coefficient is 0.0 and the mean | next: Investigate the implementation of the `construct_braided_category` function to ensure it correctly constructs braided categories associated with Boolean satisfiability instances. Once corrected, retest the conjecture using a valid implementation.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15953
2	preregistration	ollama_remote	glm4:latest	0	0	9595
3	novelty	ollama_remote	glm4:latest	0	0	8380
4	novelty	ollama_remote	glm4:latest	0	0	12677
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14967
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11027
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11956
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12233
9	critic	ollama_remote	glm4:latest	0	0	14126
10	judge	ollama_remote	glm4:latest	0	0	10556

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 121469 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/499b999bfe8f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/499b999bfe8f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/499b999bfe8f.tar.gz` (if generated)